Running π0.5 on an OpenMANIPULATOR-X: All the Ways to Run It and an End-to-End Pipeline Plan

Recommendation: run π0.5 by LoRA fine-tuning the `pi05_base` checkpoint with Physical Intelligence's official openpi JAX code on your ~24 GB GPU. Serve the result with openpi's websocket policy server. A thin ROS 2 client node, which you write, feeds the server's action chunks into your existing `arm_controller`. Keep LeRobot as your data format and as the toolchain for ACT/SmolVLA. Keep LeRobot's `pi05` policy as the second route, for asynchronous inference and real-time chunking (RTC) experiments. Running zero-shot with no fine-tuning (e.g., `pi05_droid`) is not viable on a 4-DoF OpenMANIPULATOR-X: the π0.5 paper describes PI's own robots as having "two 6 DoF arms with parallel jaw grippers", with a total state and action dimensionality of "18 or 19". It is only useful as a check that your software stack works.

TL;DR

- **Best path for you:** collect 50–200 teleoperated demos per task in LeRobotDataset format. Use a front camera plus a wrist camera, 5-D absolute joint state, and 5-D joint-position actions, trained as joint deltas with an absolute gripper. LoRA fine-tune `pi05_base` in openpi JAX; the README gives >22.5 GB for LoRA, so a 24 GB card is just enough. Serve it with `serve_policy.py` on port 8000. A ROS 2 node buffers each action chunk and streams it to `/arm_controller` behind a safety layer.
- **The other ways to run π0.5:**
  - openpi PyTorch: full fine-tuning only, with no LoRA, FSDP, mixed precision or EMA, so it is impractical on 24 GB.
  - LeRobot `pi05`: on 24 GB you must train only the action expert or use PEFT. In exchange you get bf16, gradient checkpointing, async inference and RTC.
  - Zero-shot `pi05_droid` / `pi05_libero`: a debugging aid only.
  - RL post-training with RLinf: a later stage, and it needs much more compute.
  - Cloud GPUs (A100/H100) for full fine-tuning.
  - TensorRT/Jetson-class edge deployment: still community or vendor work, not turnkey.
- **The biggest risks:** normalization statistics that differ between training and serving; camera keys or orientation that differ between training and serving; mistakes in gripper conventions and in the delta/absolute action setting; running out of memory on 24 GB; and unsafe action chunks on a small arm whose motors overload easily. Your own progress report (7 Oct 2026) already records "Hardware Error Status = 32 (overload)" on the XM430-W350 motors. Budget your time around these, not around model choice.

Key Findings

1. **openpi is the reference implementation, and its JAX LoRA path is the only officially documented route that fits ~24 GB.** The openpi README lists these requirements, each tested on an RTX 4090 or A100/H100:
   - inference: >8 GB

- LoRA fine-tuning: >22.5 GB
- full fine-tuning: >70 GB

It ships three π0.5 checkpoints: `pi05_base` (for fine-tuning), `pi05_droid` and `pi05_libero`. The README also says openpi currently supports only the flow-matching (action) head of π0.5 for training and inference. The model's high-level subtask text prediction is not exposed. mcp__Google_Drive__read... Your hierarchical "planner" layer therefore has to be external, for example a VLM that writes the prompt π0.5 receives. That fits your surprise-monitor design well.

2. **openpi's PyTorch port (September 2025) is for full-parameter training and for inference.** It converts JAX checkpoints with `examples/convert_jax_model_to_pytorch.py` and needs patched `transformers` files. The README lists as unsupported: π0-FAST, mixed-precision training, FSDP, LoRA and EMA. For a 24 GB card the PyTorch port is useful mainly for inference with `torch.compile`, not for training.
3. **LeRobot's `pi05` policy brings everything into one toolchain:**
   - commands: `lerobot-record`, `lerobot-train`, and async/remote inference
   - quantile normalization by default
   - training flags: `--policy.dtype=bfloat16`, `--policy.gradient_checkpointing=true`, `--policy.compile_model=true`
   - RTC for flow-matching policies

   Full fine-tuning of the model does not fit in 24 GB with AdamW. The π0.5 paper (arXiv:2504.16054) gives its size as "SigLIP (400M) + Gemma (2.6B)" plus an "action expert (300M)". On your card, train only the action expert or use PEFT/LoRA.
4. **The OpenMANIPULATOR-X is not a LeRobot robot, so you need a bridge either way.** Both frameworks consume LeRobot-format datasets. The work you cannot avoid is two things:
   - converting rosbag2 recordings to a LeRobot dataset, or writing a LeRobot robot plugin
   - writing a ROS 2 action-execution node

   Neither framework provides these for your arm.
5. **π0.5's model interface:**
   - images: up to 3 views at 224×224 (base, left wrist, right wrist); a missing view is masked out
   - state and action: zero-padded to 32 dimensions
   - normalization: per the π0.5 paper, "All action data is normalized to [−1, 1] using the 1% and 99% quantile of each action dimension of the individual dataset", and "For robots with lower-dimensional configuration and action spaces, we zero-pad the action vectors"
   - prompt: a text instruction
   - output: action chunks of 50 steps by default (the paper uses "an action horizon of 50", generated with "10 denoising steps" and commanded "at 50 Hz (with action chunking)"; openpi's LIBERO config uses 10)

   The π0.5 paper states that "The robot proprioceptive state is discretized and input to the

model as text tokens." Your 5-D arm uses 5 of the 32 slots.

Comparison of All Ways to Run π0.5

Method	What it is	Pros	Cons	GPU / memory	Best use for you
openpi JAX + LoRA fine-tune	Official PI code, starting from `pi05_base`. LoRA covers the 2B PaliGemma-initialized VLM ("width=2048, depth=18") and the 300M action expert ("width=1024"), per π0.5 paper Appendix E	Reference recipe; same code PI uses; documented `compute_norm_stats` → `train` → `serve_policy` workflow; websocket server decouples it from ROS	JAX/XLA memory quirks; tested on Ubuntu 22.04 only; you must write a custom `Inputs` / `Outputs` transform and TrainConfig; openpi pins an older LeRobot dataset version	>22.5 GB to train (tight on 24 GB); >8 GB to serve	**Primary training and serving path**
openpi JAX full fine-tune	Updates all parameters: SigLIP (400M) + Gemma (2.6B) + action expert (300M), per the π0.5 paper	Highest ceiling for a new embodiment	Does not fit on your card	>70 GB (A100 80GB / H100)	Rented cloud GPU, if LoRA plateaus
openpi PyTorch	PyTorch port with JAX→PyTorch conversion; `torch.compile`	PyTorch ecosystem; easier to modify (e.g., add a surprise head); fast compiled inference	No LoRA, FSDP, mixed-precision training or EMA, so training needs a big GPU; needs patched `transformers`	Inference fits; training ⩾24 GB	Inference experiments; research changes to the model
LeRobot `pi05`	Hugging Face reimplementation, weights at `lerobot/pi05_base` and `lerobot/pi05_libero`	Same stack as your ACT/ SmolVLA work; bf16 and gradient checkpointing; async policy server and robot client; RTC; Hub integration	Must freeze parts (expert-only / PEFT) on 24 GB; results can differ from openpi; your arm still needs a plugin	~24–40 GB depending on what is frozen; inference ~8–10 GB	**Second path**, and the main one for RTC/ async experiments
Zero-shot `pi05_droid` / `pi05_libero`	Run released fine-tuned checkpoints with no training	Zero cost; proves the camera → server → client loop works	Wrong embodiment (DROID is a 7-DoF Franka; LIBERO is a	>8 GB	Plumbing smoke test and LIBERO sim checks only

Method	What it is	Pros	Cons	GPU / memory	Best use for you
			simulated Franka), so the actions are meaningless on a 4-DoF arm		
FineART-VLA	Community framework on top of LeRobot π0.5 that adds a language head for task → subtask → action	Restores the hierarchical text output openpi does not expose	Research-grade; small user base; maturity unverified	Similar to LeRobot `pi05`	Alternative to an external planner if you want subtask prediction built in
RLinf (RL post-training)	RL framework with π0/π0.5 support (PPO/GRPO) in simulation and on real robots	Can push success rates past what imitation learning gives; supports RTC	Heavy multi-GPU setup; needs a simulator or a reset-able real setup and a reward	Multi-GPU (A100/H100 class)	Final stage, after a working supervised policy
Local inference (same PC as ROS)	Policy server and ROS 2 on one machine	Lowest latency; simplest	GPU shared with any simulation; dependency conflicts (Python 3.11 vs Jazzy's 3.12), solved with Docker or a separate uv env	≥8 GB free	**Default for you**
Remote policy server (LAN)	openpi websocket or LeRobot gRPC policy server on a GPU box; robot PC runs the client	Robot PC can be a laptop; easy to swap GPUs	Network latency and jitter; needs async execution	Server-side only	If the robot PC is not the GPU machine
Cloud training (rented A100/H100, HF Jobs, Colab)	Train elsewhere, download the checkpoint, serve locally	Allows full fine-tuning; faster experiments	Cost; moving data and checkpoints; Colab sessions are unreliable for multi-hour	40–80 GB	Full fine-tuning or large batches later

Method	What it is	Pros	Cons	GPU / memory	Best use for you
			runs		
Docker vs uv	openpi supports both; ROS 2 already runs in Docker	Docker isolates CUDA, Python and Ubuntu version mismatches (you're on 24.04)	GPU passthrough setup (`nvidia-container-toolkit`)	—	Put openpi in its own container, separate from the ROS 2 container
TensorRT / ONNX / quantization / Jetson	NVIDIA and community efforts to accelerate π0-family inference	Large latency cuts are reported for π0 on an RTX 4090	No official openpi TensorRT export; fragile; Jetson Orin memory is tight	Edge devices	Not needed for a desktop 3090/4090; skip
Simulation (LIBERO, LIBERO-Plus, MuJoCo OM-X, Gazebo, LeIsaac)	Test in simulation before the real robot	Safe; reproducible; LIBERO has published π0.5 numbers to reproduce	Sim-to-real gap; OM-X has no ready-made π0.5 sim task	Same as inference	LIBERO to validate the install; MuJoCo/ Gazebo OM-X to test the ROS client

Details: Step-by-Step Pipeline Plan

Step 0: Environment

- Check the real GPU with `nvidia-smi`. "26 GB" is almost certainly a 24 GiB card. If it is an RTX 3090, expect roughly 1.5–2× the 4090's latency and training time.
- openpi install, either way:
  - with uv: `git clone --recurse-submodules https://github.com/Physical-Intelligence/openpi.git && GIT_LFS_SKIP_SMUDGE=1 uv sync && GIT_LFS_SKIP_SMUDGE=1 uv pip install -e .`
  - or with the repo's Docker setup
- openpi is tested on Ubuntu 22.04. Your host is 24.04 and ROS 2 Jazzy runs in Docker, so give openpi a separate Ubuntu 22.04-based container with `--gpus all` and `--network host`. mcp__Google_Drive__read...
- Install only the lightweight `openpi-client` package (`packages/openpi-client`) inside the ROS 2 container. It needs numpy, msgpack and websockets, not JAX.
- LeRobot install: `pip install -e ".[pi]"` in a third environment, used for ACT, SmolVLA and LeRobot `pi05`.

Step 1: Hardware additions

- **Cameras:**
  - one fixed front/overhead camera (e.g., Logitech C920 or Intel RealSense D405/D435 in RGB-only mode) covering the whole workspace
  - one wrist camera (a small USB camera mounted on link 5 near the gripper)
  - fix the exposure and white balance
  - record at 640×480, 30 fps (π0.5 resizes to 224×224 anyway)
  - never move the front camera between data collection and evaluation
- **Teleoperation leader:** best to worst:
  1. A second OpenMANIPULATOR-X with torque disabled, used as the leader. It has identical kinematics, so the joint mapping is 1:1.
  2. A GELLO-style 3D-printed leader with XL330 servos, matched to OM-X link lengths.
  3. Keyboard, gamepad or MoveIt servo teleoperation. These give jerkier, less human-like demos, which hurts imitation learning.

  Use the leader arm if your budget allows. Demo quality matters more than anything else.
- **Safety hardware:** a physical e-stop that cuts motor power, plus Dynamixel Current Limit set lower than the factory value. This protects you from the overload you already hit with the default init pose. mcp__Google_Drive__read...

Step 2: Data collection

Option A: **rosbag2 → LeRobotDataset converter.** Recommended, because it reuses your ROS stack.

- Record:
  - `/joint_states` (position, velocity, effort/current)
  - the commanded leader positions (a topic you publish from the leader)
  - `/arm_controller/joint_trajectory` or the controller state
  - both camera image topics, plus `camera_info`
  - an episode/task annotation topic, or a sidecar JSON
- Log for the surprise monitor too: Dynamixel Present Current, Present Velocity, Goal Position and Present Position. Make sure `dynamixel_hardware_interface` exposes current as a state interface. These go into extra dataset fields that π0.5 does not need, but your forward model does.
- Convert offline: resample every stream to a fixed grid at 15–30 Hz, using the nearest-previous message by header stamp, and reject frames with more than half a period of skew. Then call `LeRobotDataset.create(repo_id, fps, robot_type="openmanipulator_x", features=...)`, followed by `add_frame()` per step and `save_episode()` per episode. Use these features:
  - `observation.images.front` and `observation.images.wrist`: (480, 640, 3) video
  - `observation.state`: (5,) = [j1..j4 rad, gripper normalized to 0–1]
  - `action`: (5,) = the commanded leader positions at the same timestep

- `task`: the prompt string, e.g., "pick up the red cube and place it in the bowl"
- optional: `observation.current` (5,) and `observation.velocity` (5,)

Option B: **a LeRobot robot plugin** (a `lerobot_robot_openmanipulator_x` package containing a `Robot` subclass and a config). It talks to the Dynamixels directly or wraps rclpy. This gives you native `lerobot-record`, `lerobot-teleoperate` and the async robot client, but it bypasses or duplicates your ros2_control stack. Do this later only if you commit to LeRobot RTC/async.

Dataset version trap: LeRobot now writes dataset format v3.0. openpi's training pipeline has historically pinned an older LeRobot version that reads the v2.x format. Before collecting much data, check which version your openpi checkout loads. If needed, write a second copy of the data with openpi's own pinned `lerobot`, or convert it.

Recording tips:

- Start each episode from a consistent home pose.
- Trim idle frames at the start and end, because idle frames teach the policy to stall.
- Vary object positions.
- Keep the task string identical across episodes of the same task.

Step 3: Action and state space design

- **State:** absolute joint positions in radians for joints 1–4, plus the gripper mapped to [0, 1] (0 = open, 1 = closed, matching the convention most π datasets use). Write that convention down and test it.
- **Action:** record absolute target joint positions. In openpi, apply the `DeltaActions` transform to joints 1–4 only, with `make_bool_mask(4, -1)`, and `AbsoluteActions` on output. This follows the openpi UR5 and ALOHA examples, which make joint actions relative to the current state while the gripper stays absolute. Do not use end-effector actions: a 4-DoF arm cannot reach arbitrary 6-D poses, and IK in the loop adds failure modes.
- **Padding:** openpi pads state and action to `action_dim=32`. Your output transform must slice `actions[:, :5]`.
- **Normalization:** π0.5 uses quantile normalization. Run `compute_norm_stats` on your own dataset. The OM-X is not in the pretraining mixture, so do not reuse the base model's normalization statistics. mcp__Google_Drive__read... The same `norm_stats.json` must travel with the checkpoint into serving; openpi stores it in the checkpoint's assets.
- **Timing:**
  - control and record at 15–30 Hz
  - chunk length (`action_horizon`): 10 at 15 Hz (like `pi05_libero`) or up to 50 at 30 Hz (π0.5's default) mcp__Google_Drive__read...
  - execute 5–25 steps open-loop before re-querying the model

  Shorter execution horizons react faster, which helps your surprise monitor, at the cost of more inference calls.

Step 4: openpi data config and policy transforms

- Copy `src/openpi/policies/libero_policy.py` to `omx_policy.py`.

- `OMXInputs`: map `observation/front_image` → `base_0_rgb`, `observation/wrist_image` → `left_wrist_0_rgb`, and fill `right_wrist_0_rgb` with zeros and mask = False. Pass `state` (5,) and `prompt` through.
- `OMXOutputs`: return `actions[:, :5]`.

- In `src/openpi/training/config.py`, add `LeRobotOMXDataConfig`, modeled on `LeRobotLiberoDataConfig`. Give it a `RepackTransform` that maps your dataset keys to the policy keys, and the delta-action transforms from Step 3.
- Add `TrainConfig(name="pi05_omx_lora", ...)` with:
  - `model=pi0_config.Pi0Config(pi05=True, action_horizon=<H>, paligemma_variant="gemma_2b_lora", action_expert_variant="gemma_300m_lora")`
  - `freeze_filter` from the same model config's `get_freeze_filter()`
  - `ema_decay=None`
  - `weight_loader=CheckpointWeightLoader("gs://openpi-assets/checkpoints/pi05_base/params")`
  - `batch_size` of 8–32
  - `num_train_steps` of 20k–30k

  This mirrors the repo's `pi0_libero_low_mem_finetune` LoRA recipe with `pi05=True`.

Step 5: Fine-tuning on 24 GB

1. `uv run scripts/compute_norm_stats.py --config-name pi05_omx_lora`
2. `XLA_PYTHON_CLIENT_MEM_FRACTION=0.9 uv run scripts/train.py pi05_omx_lora --exp-name=omx_v1 --overwrite`
3. If you run out of memory:
   - lower `batch_size` (8, then 4)
   - set `XLA_PYTHON_CLIENT_MEM_FRACTION=0.95`
   - close every other GPU process, including RViz, Gazebo and the display server if possible
   - keep EMA off
   - reduce to 2 cameras (already done)

   `fsdp_devices` helps only with more than one GPU.
4. **How many demos:** 50 good demos per task is a sensible minimum; 100–200 if you want robustness to object placement. Language-conditioned multitask runs need more.
5. **Time:** 20–30k LoRA steps on a single 4090 should take several hours to about a day; a 3090 will be slower. Save every 5k steps, then choose the checkpoint by success rate on the real robot, not by training loss. Flow-matching loss correlates poorly with success.
6. **LeRobot alternative:** `lerobot-train --policy.type=pi05 --policy.pretrained_path=lerobot/pi05_base --dataset.repo_id=<you>/omx_pick --policy.dtype=bfloat16 --policy.gradient_checkpointing=true --policy.train_expert_only=true --batch_size=8 --steps=20000`. Add `--policy.compile_model=true` once the run is stable. If your dataset has no quantile statistics, run LeRobot's quantile-stats augmentation script, or switch normalization to MEAN_STD. Do this consistently.

Step 6: Serving and the ROS 2 execution node

- **Server:** `uv run scripts/serve_policy.py policy:checkpoint --policy.config=pi05_omx_lora --policy.dir=checkpoints/pi05_omx_lora/omx_v1/<step>`. It listens for websockets on port 8000.
- **Client node (`pi05_bridge`, rclpy):**
  1. Subscribe to `/joint_states` and both camera topics. Keep the latest of each, with timestamps. mcp__Google_Drive__read...
  2. On each inference tick, build `{"observation/front_image": uint8 224×224, "observation/wrist_image": ..., "observation/state": float32[5], "prompt": str}`. Use `openpi_client.image_tools.resize_with_pad`, with exactly the same preprocessing as in the converter.
  3. Call `WebsocketClientPolicy("localhost", 8000).infer(obs)["actions"]` to get an (H, 5) chunk.
  4. Push the chunk into a buffer. A timer at the control rate pops one action per tick, passes it through the safety layer, and publishes it. Use either `trajectory_msgs/JointTrajectory` on `/arm_controller/joint_trajectory`, with one point and `time_from_start` = 1–2 control periods, or the executed slice of the chunk as a multi-point trajectory with `time_from_start = k/fps`. The JointTrajectoryController splines between points and replaces the old trajectory when a new one arrives. Send the gripper through `gripper_controller`, mapped from 0–1 back to joint units. mcp__Google_Drive__read...
  5. **Asynchronous execution:** request the next chunk when `n_remaining` falls to about the inference latency × fps plus a margin. To start, cross-fade overlapping chunks. Later, replace the cross-fade with RTC (LeRobot) for smoother joins.
- **Latency:** π0-family inference with two views on an RTX 4090 runs from tens of milliseconds to roughly 100 ms per chunk. That is fine for 15–30 Hz control with chunking.
- **Safety layer (non-negotiable):**
  - clamp to URDF joint limits minus a margin
  - limit per-step Δq, giving a maximum joint velocity
  - abort on Present Current above a threshold
  - workspace box check through forward kinematics
  - watchdog: hold position if no fresh chunk arrives within 2× the expected latency, or if the camera or joint states are stale
  - software e-stop topic, plus the hardware e-stop
  - launch with `init_position:=false`, which you already found necessary mcp__Google_Drive__read...

  Your surprise monitor plugs in here. It compares predicted and measured current and position each tick, and on a large residual it flushes the buffer, holds, and asks the planner to retry or re-prompt.

Step 7: Evaluation and debugging

- **Open-loop replay first:** feed recorded dataset frames to the server and plot predicted vs. recorded actions. This catches errors in normalization, keys, delta/absolute settings

and gripper sign before the robot moves.

- **Then "shadow mode":** run the policy live without sending commands, and visualize the commands in RViz.
- **Then low-speed real execution:** velocity limits at 25%, then increase.
- **Protocol:**
  - 20 trials per task
  - fixed randomized object placements
  - report success rate, time to success and interventions
  - ablate the camera set and the execution horizon
  - report surprise-triggered retries separately

Pipeline Architecture

1. Teleop and record. The leader arm and two cameras feed ROS 2 topics into rosbag2.

2. Convert. The rosbag2 converter (or a LeRobot plugin) produces a LeRobotDataset at a fixed fps with front and wrist images, 5-D state, 5-D action, task text, and extra current/velocity fields.

3. Train. Build normalization statistics, then train ACT or SmolVLA in LeRobot as baselines. π0.5 LoRA trains in openpi (or LeRobot `pi05` expert-only). Checkpoints and `norm_stats` are stored together.

4. Serve. The openpi server runs in Docker on the GPU and receives observations from the ROS 2 `pi05_bridge` over websockets on port 8000.

5. Execute. `pi05_bridge` keeps a chunk buffer and runs it through the safety layer, with the surprise monitor alongside. Commands go to `arm_controller` and `gripper_controller` in ros2_control, then to the U2D2 and the Dynamixels. mcp__Google_Drive__read...

6. Evaluate. Success-rate trials, intervention logs and surprise logs feed back into new data collection and the hierarchical planner, which sends new prompts and retries to π0.5.

Staged Timeline (about 12–14 weeks)

Week	Stage	Exit criterion
1	openpi install; run `pi05_libero` in LIBERO sim; zero-shot `pi05_droid` via the websocket client	Reproduce close to the published LIBERO numbers; end-to-end client loop works
2	Fix Gazebo controller init or use MuJoCo OM-X; build `pi05_bridge` with a dummy policy (sinusoid) plus the safety layer	Safe streaming of chunks to `arm_controller` in sim and on the real arm
3–4	Cameras, leader arm, recorder, rosbag→LeRobot converter; 50 demos of one task	Dataset loads in LeRobot and openpi; replay visualizer looks correct
5	ACT baseline, then SmolVLA, in LeRobot	ACT reaches ≥50% on the single task (this validates the data pipeline)
6–7	π0.5 LoRA in openpi; open-loop replay test; shadow mode; real trials	π0.5 matches or beats ACT; norm/key/gripper bugs fixed
8–9	Multitask/language variations; 100–200 demos; LeRobot `pi05` comparison with async and RTC	Language following across 2–3 tasks
10–12	Surprise forward model (current/position), retry and re-prompt logic, external high-level planner	Measurable recovery gain on perturbed trials
13+	Optional: RLinf RL post-training in sim, or a cloud full fine-tune	Only if time allows

Risks, Pitfalls and Mitigations

- **Normalization mismatch:**
  - forgetting to rerun `compute_norm_stats` after adding data
  - serving with the wrong assets
  - mixing QUANTILES and MEAN_STD between LeRobot and openpi

  Mitigation: version the norm stats with the dataset hash.
- **Camera key or orientation errors:** swapped front/wrist, BGR vs RGB, CHW vs HWC, flipped images. Mitigation: the replay test, and saving the preprocessed 224×224 inputs from the server.
- **Gripper convention:** inverted open/closed, or unnormalized units, gives a policy that never grasps. Mitigation: unit-test the mapping in both directions.
- **Delta/absolute confusion:** applying `DeltaActions` in training but not reversing it in serving, or vice versa, gives drift.
- **Idle frames and pauses** in demos give a policy that hesitates. Trim them.
- **OOM on 24 GB:** JAX preallocation plus a desktop compositor plus RViz can push LoRA over the limit. Run training headless and use small batches.
- **Dataset format drift** (LeRobot v3 vs openpi's pinned version): verify on day one.
- **Hardware overload:** chunks with large Δq can stall the XM430s. Mitigate with velocity

clamps, a lower current limit and the watchdog. mcp__Google_Drive__read...

- **Over-expecting zero-shot:** no released π0.5 checkpoint knows your embodiment. Fine-tuning is required.
- **Python/ROS conflicts:** keep JAX/openpi out of the ROS environment entirely. Only `openpi-client` crosses over.

Caveats

- **Live web access:** the web-search tooling was not available during this session. Version-specific details (exact LeRobot release numbers in 2026, PEFT flag names, FineART-VLA maturity, TensorRT/Jetson status, current openpi config names) come from the openpi and LeRobot documentation as last known, and from your Drive files. Check them against the live repositories before you rely on them. In particular, check whether openpi has since added an official π0.5 LoRA config, RTC, or π0.6 weights. π0.6/π*0.6 was announced by Physical Intelligence in late 2025, but open weights had not been confirmed when this was compiled.
- **Training time and latency** figures are order-of-magnitude estimates for a 4090-class GPU, not benchmarks on your machine.
- **Your research goal:** π0.5's flow-matching-only availability in openpi actually helps it. The low-level policy stays a clean, frozen executor, and your contribution sits in the surprise monitor and the planner.